

Assignment No 9

Name: Sciddhanto Sinha

Class: CSE, SY-2(B)

Roll No.:2213111

Enrollment No.: MITU21BTCS0553

Aim: (Functions)

Write a program to build the animal shelter (holding cats, dogs, and rabbit) operations using Python class. People can adopt the animals based on their admission time. Create a data structure to perform various operations such as - new entry in the shelter, adopt a dog, adopt a cat, adopt a rabbit, adopt any animal, display the shelter information

Theory:

A class is a user-defined blueprint or prototype from which objects are created. Classes provide a means of bundling data and functionality together. Creating a new class creates a new type of object, allowing new instances of that type to be made. Each class instance can have attributes attached to it for maintaining its state. Class instances can also have methods (defined by their class) for modifying their state.

Syntax: Class Definition

```
class ClassName:
```

```
    # Statement
```

Syntax: Object Definition

```
obj = ClassName()
```

```
print(obj.attr)
```

Class creates a user-defined data structure, which holds its own data members and member functions, which can be accessed and used by creating an instance of that class. A class is like a blueprint for an object.

Defining a class

```
# Python3 program to
```

```
# demonstrate defining
```

```
# a class
```

```
class Dog:
```

```
    pass
```

In the above example, the class keyword indicates that you are creating a class followed by the name of the class (Dog in this case).

Class Objects

An Object is an instance of a Class. A class is like a blueprint while an instance is a copy of the class with actual values. It's not an idea anymore, it's an actual dog, like a dog of breed pug who's seven years old. You can have many dogs to create many different instances, but without the class as a guide, you would be lost, not knowing what information is required.

An object consists of :

State: It is represented by the attributes of an object. It also reflects the properties of an object.

Behaviour: It is represented by the methods of an object. It also reflects the response of an object to other objects.

Identity: It gives a unique name to an object and enables one object to interact with other objects.

Declaring an object

```
# Python3 program to  
# demonstrate instantiating  
# a class
```

```
class Dog:
```

```
    # A simple class
```

```
    # attribute
```

```
    attr1 = "mammal"
```

```
    attr2 = "dog"
```

```
# A sample method
def fun(self):
    print("I'm a", self.attr1)
    print("I'm a", self.attr2)
```

```
# Driver code
# Object instantiation
Rodger = Dog()
```

```
# Accessing class attributes
# and method through objects
print(Rodger.attr1)
Rodger.fun()
```

```
mammal
I'm a mammal
I'm a dog
```

The `__init__` method is similar to constructors in C++ and Java. Constructors are used to initialize the object's state. Like methods, a constructor also contains a collection of statements(i.e. instructions) that are executed at the time of Object creation. It runs as soon as an object of a class is instantiated. The method is useful to do any initialization you want to do with your object.

```
# Sample class with init method
class Person:
```

```
    # init method or constructor
    def __init__(self, name):
        self.name = name
```

```
# Sample Method
```

```
def say_hi(self):  
    print('Hello, my name is', self.name)
```

```
p = Person('Nikhil')  
p.say_hi()
```

Output:

Hello, my name is Nikhil

An abstract class can be considered as a blueprint for other classes. It allows you to create a set of methods that must be created within any child classes built from the abstract class. A class which contains one or more abstract methods is called an abstract class.

```
from abc import ABC, abstractmethod
```

```
class Polygon(ABC):
```

```
    @abstractmethod  
    def noofsides(self):  
        pass
```

```
class Triangle(Polygon):
```

```
    # overriding abstract method  
    def noofsides(self):  
        print("I have 3 sides")
```

```
class Pentagon(Polygon):
```

```
    # overriding abstract method  
    def noofsides(self):  
        print("I have 5 sides")
```

```
class Hexagon(Polygon):  
  
    # overriding abstract method  
    def noofsides(self):  
        print("I have 6 sides")
```

```
class Quadrilateral(Polygon):  
  
    # overriding abstract method  
    def noofsides(self):  
        print("I have 4 sides")
```

```
# Driver code  
R = Triangle()  
R.noofsides()  
K = Quadrilateral()  
K.noofsides()  
R = Pentagon()  
R.noofsides()  
K = Hexagon()  
K.noofsides()
```

Output:

```
I have 3 sides  
I have 4 sides  
I have 5 sides  
I have 6 sides
```

Program:

```
#in file animalinfo.py
from AnimalShelterServiceImpl import AnimalShelterServiceImpl

class Animal:

    def __init__(self, aid, name, age, type, owner):
        self.animal_id = aid
        self.name = name
        self.age = age
        self.type = type
        self.owner = owner

    def __str__(self):
        return f'id : {self.animal_id}, name : {self.name}, age : {self.age}, type:{self.type}, '\
            f'owner : {self.owner}'

a1 = Animal(1, "Moti", 4, "dog", "")
a2 = Animal(2, "Lucy", 2, "cat", "")
a3 = Animal(3, "tyler", 1, "dog", "")
```

```
a4 = Animal(4, "Mani", 1.2, "cat", "")
a5 = Animal(5, "Peter", 4, "rabbit", "")
a6 = Animal(6, "Roger", 0.6, "rabbit", "")
a7 = Animal(7, "Anaya", 3, "cow", "")
a8 = Animal(8, "Devi", 10, "cow", "")
```

```
impl = AnimalShelterServiceImpl()
impl.add_animal(a1)
impl.add_animal(a2)
impl.add_animal(a3)
impl.add_animal(a4)
impl.add_animal(a5)
impl.add_animal(a6)
impl.add_animal(a7)
impl.add_animal(a8)
```

```
impl.show_all_animals()
```

```
impl.adopt_dog("Shyam")
impl.adopt_cat("Aman")
impl.adopt_rabbit("Smita")
impl.adopt_other_animals("Raju")
print("_____")
print("_____")
impl.show_all_animals()
```

```
#in file Animalshelter.py
from abc import ABC, abstractmethod
```

```
class AnimalShelterService(ABC):
```

```
@abstractmethod
def add_animal(self, animal):
    pass
```

```
@abstractmethod
def adopt_dog(self, owner):
    pass
```

```
@abstractmethod
def adopt_cat(self, owner):
    pass
```

```
@abstractmethod
def adopt_rabbit(self, owner):
    pass
```

```
@abstractmethod
def adopt_other_animals(self, owner):
    pass
```

```
@abstractmethod
def show_all_animals(self):
    pass
```

```
#in file animalshelterimpl.py
```

```
from AnimalShelterService import AnimalShelterService
```

```
class AnimalShelterServiceImpl(AnimalShelterService):
```

```
    list_of_animal = []
```



```

def add_animal(self, animal):
    self.list_of_animal.append(animal)

def adopt_dog(self, owner):
    for i in self.list_of_animal:
        if i.type == 'dog' and i.age >= 3:
            i.owner = owner

def adopt_cat(self, owner):
    for i in self.list_of_animal:
        if i.type == 'cat' and i.age >= 2:
            i.owner = owner

def adopt_rabbit(self, owner):
    for i in self.list_of_animal:
        if i.type == 'rabbit' and i.age >= 1:
            i.owner = owner

def adopt_other_animals(self, owner):
    for i in self.list_of_animal:
        if i.type == 'cow' and i.age >= 5:
            i.owner = owner

def show_all_animals(self):
    for i in self.list_of_animal:
        print(i)

```

Output:

```

id : 1, name : Moti, age : 4, type:dog,owner :
id : 2, name : Lucy, age : 2, type:cat,owner :
id : 3, name : tyler, age : 1, type:dog,owner :

```

id : 4, name : Mani, age : 1.2, type:cat,owner :
id : 5, name : Peter, age : 4, type:rabbit,owner :
id : 6, name : Roger, age : 0.6, type:rabbit,owner :
id : 7, name : Anaya, age : 3, type:cow,owner :
id : 8, name : Devi, age : 10, type:cow,owner :

id : 1, name : Moti, age : 4, type:dog,owner : Shyam
id : 2, name : Lucy, age : 2, type:cat,owner : Aman
id : 3, name : tyler, age : 1, type:dog,owner :
id : 4, name : Mani, age : 1.2, type:cat,owner :
id : 5, name : Peter, age : 4, type:rabbit,owner : Smita
id : 6, name : Roger, age : 0.6, type:rabbit,owner :
id : 7, name : Anaya, age : 3, type:cow,owner :
id : 8, name : Devi, age : 10, type:cow,owner : Raju

Conclusion: We have successfully learnt how to create, define and use classes in python.